

Titanic.csv

Introduction:

The Titanic dataset is one of the most famous datasets in data science and is often used for practicing classification problems. It contains data about passengers aboard the Titanic, which sank during its maiden voyage in April 1912 after colliding with an iceberg.

Key Features of the Titanic Dataset:

1. **PassengerId**: A unique identifier for each passenger.
2. **Survived**: Survival status (0 = Did not survive, 1 = Survived).
3. **Pclass**: Passenger class (1 = First class, 2 = Second class, 3 = Third class).
4. **Name**: Name of the passenger.
5. **Sex**: Gender of the passenger.
6. **Age**: Age of the passenger (in years). Some values may be missing.
7. **SibSp**: Number of siblings/spouses aboard the Titanic.
8. **Parch**: Number of parents/children aboard the Titanic.
9. **Ticket**: Ticket number.
10. **Fare**: Passenger fare.
11. **Cabin**: Cabin number. This feature has many missing values.
12. **Embarked**: Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton).

1::Importing library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, roc_curve, auc
```

2::Load the data

```
df = pd.read_csv('titanic.csv')
print(df)
```

t1.py

	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
0	1	0	3 ...	7.2500	NaN	S	

1	2	1	1 ...	71.2833	C85	C
2	3	1	3 ...	7.9250	NaN	S
3	4	1	1 ...	53.1000	C123	S
4	5	0	3 ...	8.0500	NaN	S
..	...	...	...	...	...	...
886	887	0	2 ...	13.0000	NaN	S
887	888	1	1 ...	30.0000	B42	S
888	889	0	3 ...	23.4500	NaN	S
889	890	1	1 ...	30.0000	C148	C
890	891	0	3 ...	7.7500	NaN	Q

Display the first few rows:

```
print('First few rows:\n', df.head())
```

First few rows:

	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
0	1	0	3 ...	7.2500	NaN	S	
1	2	1	1 ...	71.2833	C85	C	
2	3	1	3 ...	7.9250	NaN	S	
3	4	1	1 ...	53.1000	C123	S	
4	5	0	3 ...	8.0500	NaN	S	

Check dataset info:

```
print('\nDataset Information:\n', df.info())
```

Dataset Information

None

Check missing values:

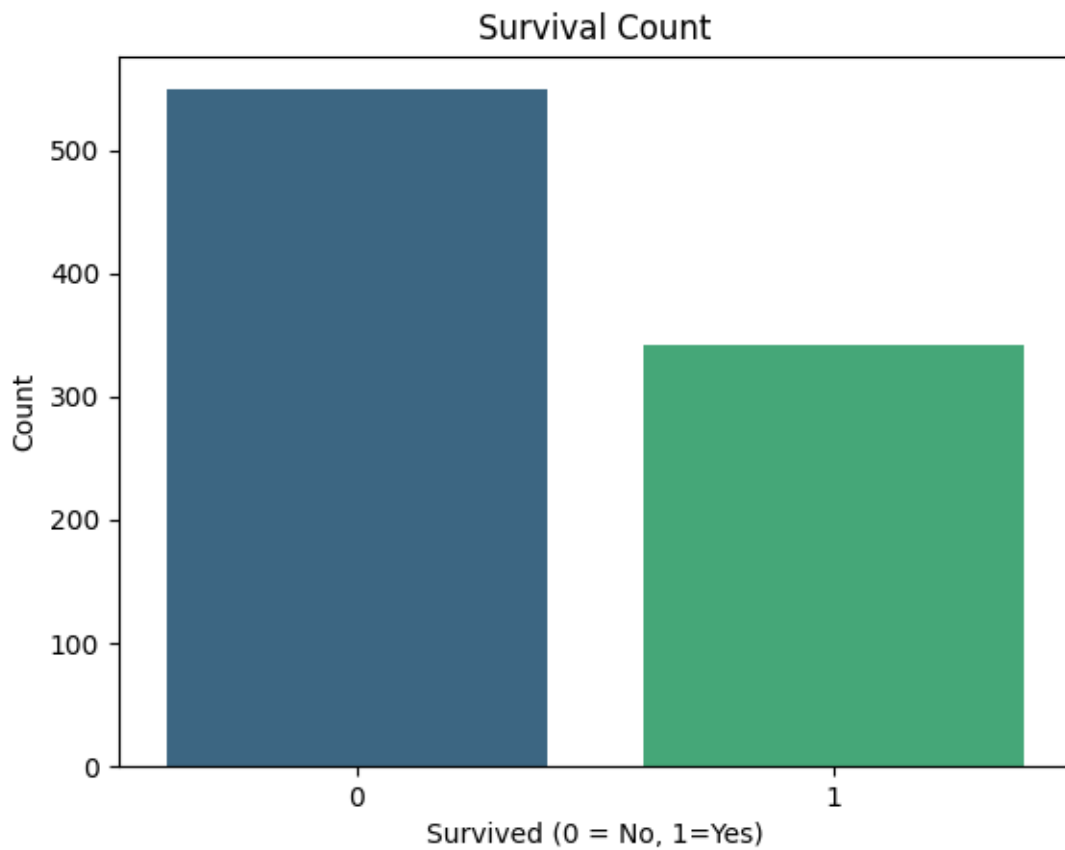
```
print('\nMissing Values:\n', df.isnull().sum())
```

Missing Values:

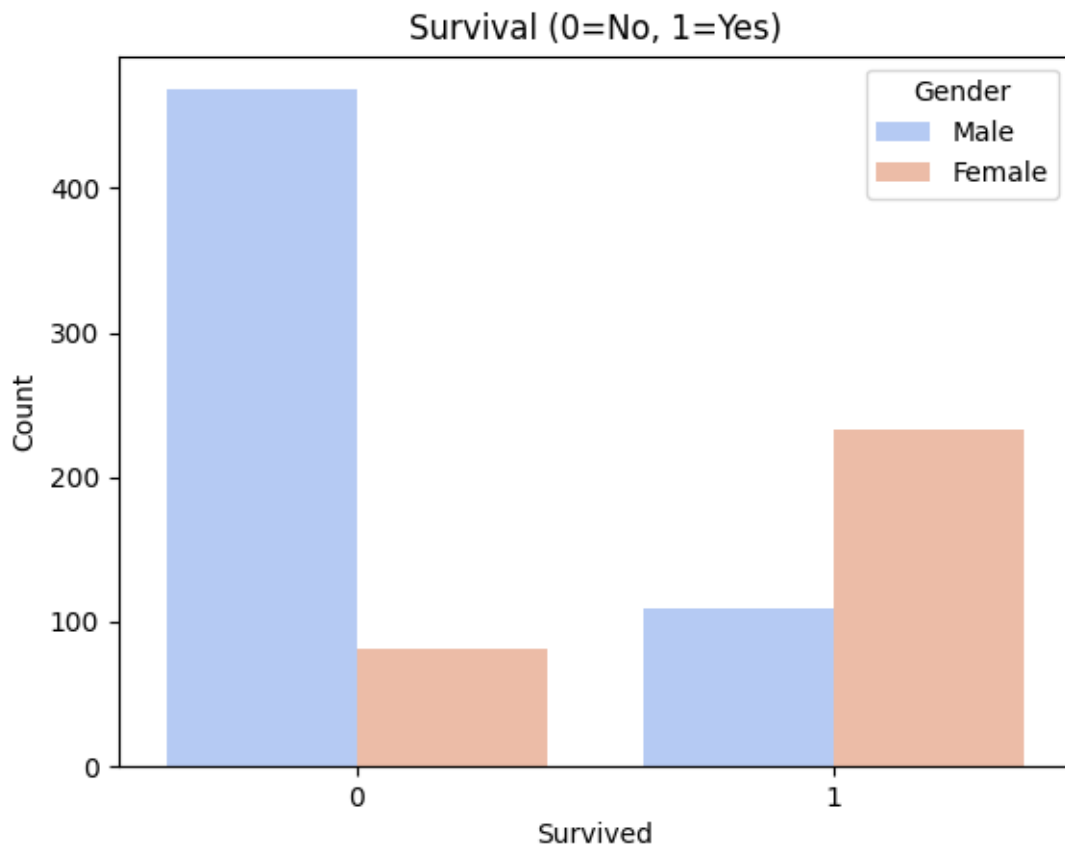
```
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked        2
dtype: int64
```

3::Exploratory Data Analysis

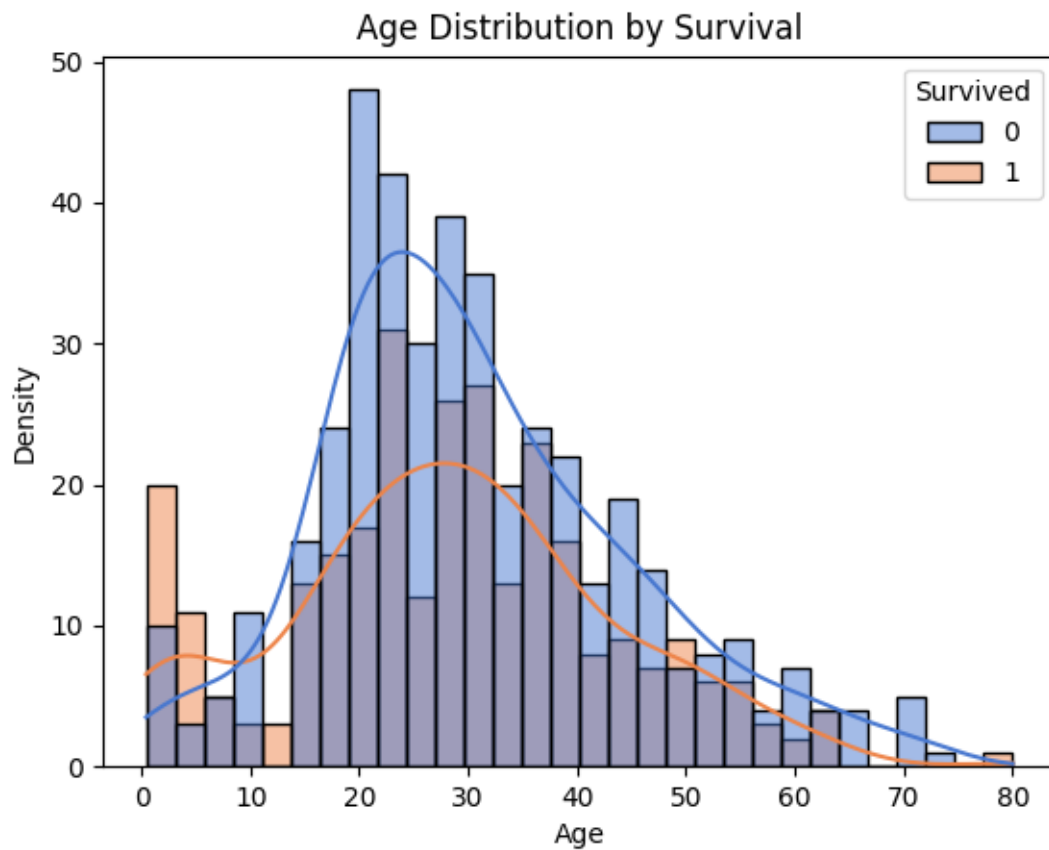
```
# Survival Distribution
sns.countplot(data=df, x='Survived', palette='viridis')
plt.title('Survival Count')
plt.xlabel('Survived (0 = No, 1=Yes)')
plt.ylabel('Count')
plt.show()
# Survival Distribution shows the ratio of survivors vs. non-survivors.
```



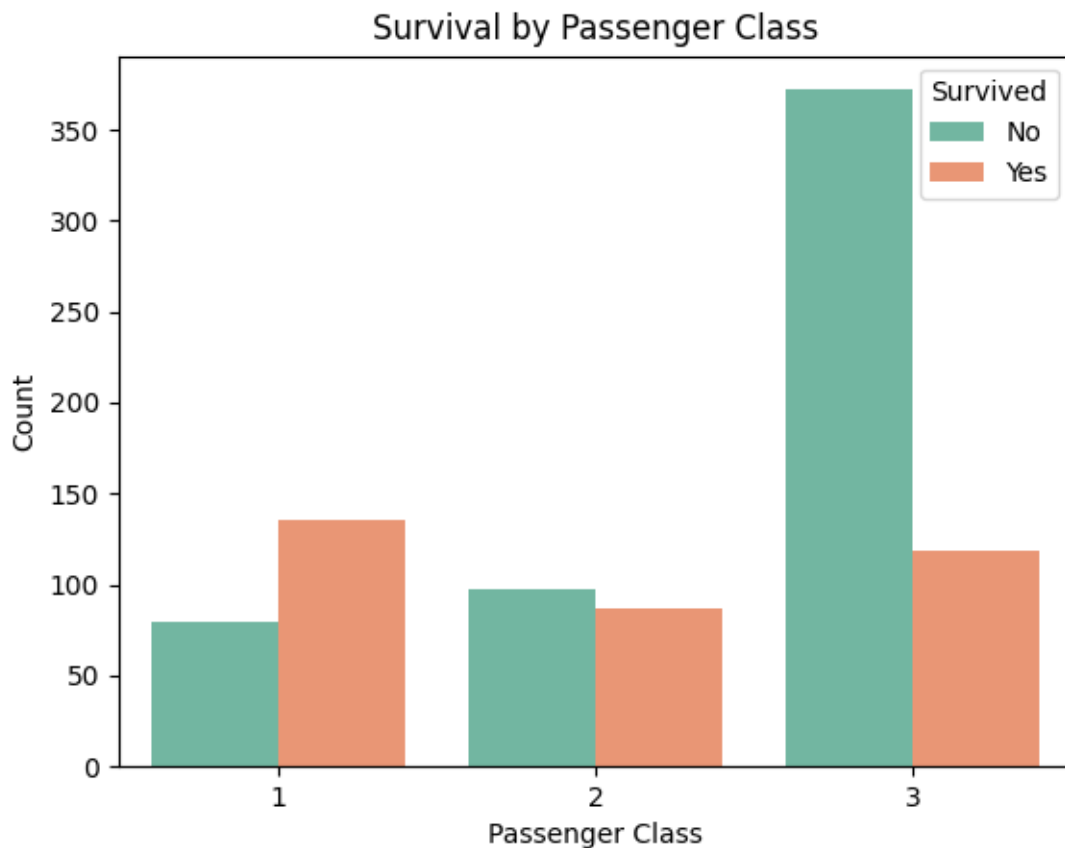
```
# Survival by Gender
sns.countplot(data=df, x='Survived', hue='Sex', palette='coolwarm')
plt.title('Survival (0=No, 1=Yes)')
plt.ylabel('Count')
plt.legend(title='Gender', labels=['Male', 'Female'])
plt.show()
# Survival by Gender highlights that females had higher survival rates.
```



```
# Age Distribution by Survival
sns.histplot(data=df, x='Age', hue='Survived', kde=True, bins=30,
palette='muted')
plt.title('Age Distribution by Survival')
plt.xlabel('Age')
plt.ylabel('Density')
plt.show()
# Age Distribution by Survival illustrates survival trends across different age
groups.
```



```
# Survival by Passenger Class
sns.countplot(data=df, x='Pclass', hue='Survived', palette='Set2')
plt.title('Survival by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Count')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
# Survival by Passenger Class reveals that first-class passengers had better
chances of survival.
```



4:: data Preprocessing

```
# Fill missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
# Missing values in Age and Embarked are filled with median and mode,
# respectively.

# Drop columns with excessive missing values or irrelevant for prediction
df.drop(columns=['Cabin', 'PassengerId', 'Name', 'Ticket'], inplace=True)
# The Cabin column is dropped due to excessive missing data.

# Encode categorical variables
df['Sex'] = df['Sex'].map({'male':0, 'female':1})
df['Embarked'] = df['Embarked'].map({'C':0, 'Q':1, 'S':2})
print(df['Sex'])
print(df['Embarked'])
# Categorical variables (Sex and Embarked) are encoded numerically.

# Create new features
```

```

df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
print(df['FamilySize'])
# A new feature (FamilySize) is created to indicate family connections

# Drop redundant features
df.drop(columns=['SibSp', 'Parch'], inplace=True)

```

```
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
0    0
```

```
1    1
```

```
2    1
```

```
3    1
```

```
4    0
```

```
..
```

```
886   0
```

```
887   1
```

```
888   1
```

```
889   0
```

```
890   0
```

```
Name: Sex, Length: 891, dtype: int64
```

```
0    2
```

```
1    0
```

```
2    2
```

```
3    2
```

```
4    2
```

```
..
```

```
886   2
```


887 2

888 2

889 0

890 1

Name: Embarked, Length: 891, dtype: int64

```
# Cleaned dataset
print('\nPreprocessed Dataset few rows:\n',df.head())
```

Preprocessed Dataset few rows:

	Survived	Pclass	Sex	Age	Fare	Embarked	FamilySize
0	0	3	0	22.0	7.2500	2	2
1	1	1	1	38.0	71.2833	0	2
2	1	3	1	26.0	7.9250	2	1
3	1	1	1	35.0	53.1000	2	2
4	0	3	0	35.0	8.0500	2	1

Conclusion: Preprocessing prepares the dataset for machine learning by ensuring completeness, consistency and usability.

5:: split data into Training and Testing Sets

```
# Separate features and target variable
X = df.drop('Survived', axis=1)
y = df['Survived']
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print(X_train, X_test, y_train, y_test)
# Conclusion: Splitting ensures that the model is evaluated on unseen data,
avoiding overfitting.
```

Pclass	Sex	Age	Fare	Embarked	FamilySize
--------	-----	-----	------	----------	------------

331	1	0	45.5	28.5000	2	1
733	2	0	23.0	13.0000	2	1
382	3	0	32.0	7.9250	2	1
704	3	0	26.0	7.8542	2	2
813	3	1	6.0	31.2750	2	7
..	...	...	...	...	...	...
106	3	1	21.0	7.6500	2	1
270	1	0	28.0	31.0000	2	1
860	3	0	41.0	14.1083	2	3
435	1	1	14.0	120.0000	2	4
102	1	0	21.0	77.2875	2	2

[712 rows x 6 columns] Pclass Sex Age Fare Embarked FamilySize

709	3	0	28.0	15.2458	0	3
439	2	0	31.0	10.5000	2	1
840	3	0	20.0	7.9250	2	1
720	2	1	6.0	33.0000	2	2
39	3	1	14.0	11.2417	0	2
..	...	...	...	...	...	...
433	3	0	17.0	7.1250	2	1
773	3	0	28.0	7.2250	0	1
25	3	1	38.0	31.3875	2	7
84	2	1	17.0	10.5000	2	1
10	3	1	4.0	16.7000	2	3

[179 rows x 6 columns] 331 0

733 0

382 0

704 0

813 0

..

106 1

270 0

860 0

435 1

102 0

Name: Survived, Length: 712, dtype: int64 709 1

439 0

840 0

720 1

39 1

..

433 0

773 0

25 1

84 1

10 1

Name: Survived, Length: 179, dtype: int64

6:: Train the model

```
# Train a random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
# conclusion: The model learns patterns in the training data to predict survival.

# 7. Evaluate the model
# Performance Metrics
# Predict on the test set
y_pred = model.predict(X_test)
print('\nY Prediction:\n',y_pred)
```

Y Prediction:

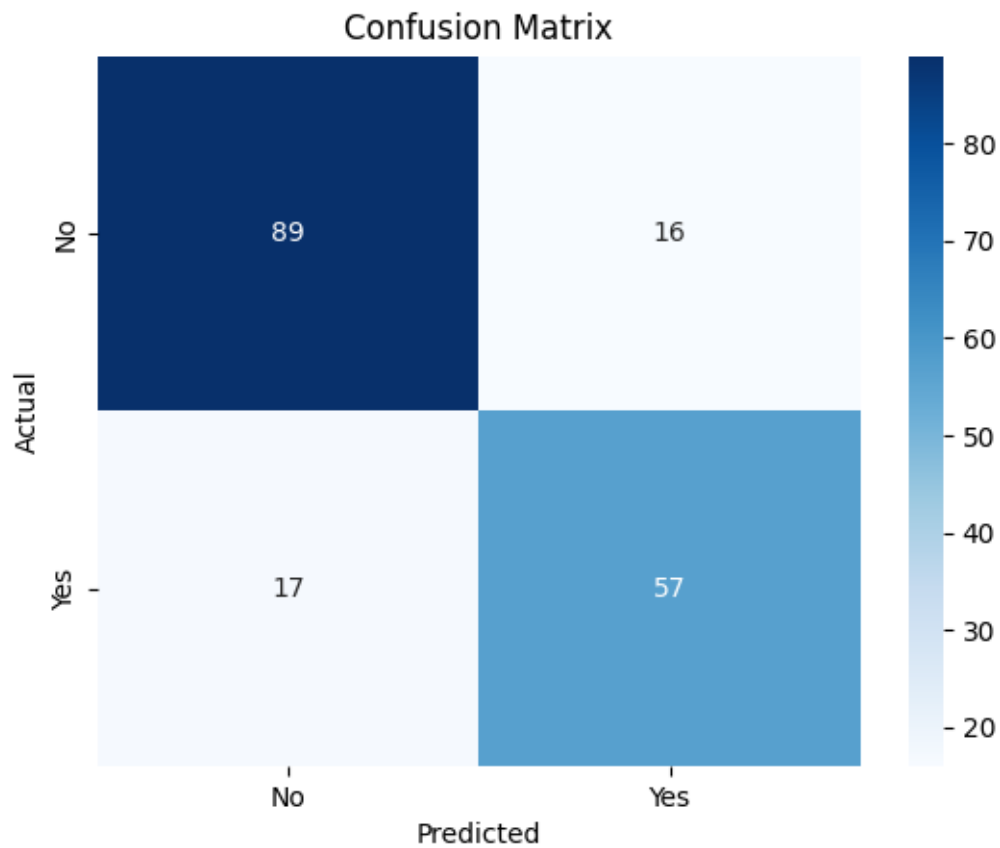
```
[0 0 0 1 0 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 1 1 1 0 0 1
 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 1 0 1 0 1 1 0 0 1 1 0 0 1 0 0 0 1 1 1 1 1
 0 0 1 1 1 1 0 1 1 0 1 0 1 1 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 1 0 1
 0 1 1 0 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 0 0 1 1 0 1 0 0 0 0 1 1 0 0 1 0 1
 1 0 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 0 1 1 1 0 0 0 1 1]
```

```
# Accuracy Score (Measures overall prediction correctness.)
print('\nAccuracy:\n', accuracy_score(y_test, y_pred))
```

Accuracy:

```
0.8156424581005587
```

```
# Confusion Matrix
# Breaks down predictions into true positives, true negatives, false positives,
and false negatives.
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No', 'Yes'],
yticklabels=['No', 'Yes'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



```
# Classification Report (Summarizes precision, recall, and F1-score.)
print('\nClassification Report:\n', classification_report(y_test, y_pred))
# Conclusion: Evaluation metrics and visualizations confirm the model's
effectiveness and highlight areas for improvement.
```

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.85	0.84	105
1	0.78	0.77	0.78	74
accuracy		0.82		179

macro avg	0.81	0.81	0.81	179
weighted avg	0.82	0.82	0.82	179

8::Insights and Conclusion

```
# important Features: analysis reveals the most significant predictors of
survival (e.g., Pclass, Sex, Age).
feature_importances = pd.DataFrame({'Feature': X.columns, 'Importance':
model.feature_importances_})
feature_importances.sort_values(by='Importance', ascending=False, inplace=True)
print("\nFeature Importances:\n", feature_importances)
```

Feature Importances:

	Feature	Importance
3	Fare	0.280461
1	Sex	0.273802
2	Age	0.253172
0	Pclass	0.079898
5	FamilySize	0.079668
4	Embarked	0.032999

Conclusion for the Titanic Dataset Analysis:

The Titanic dataset provides valuable insights into the factors that influenced survival rates during the Titanic disaster. By analyzing this dataset, we can draw meaningful conclusions and practice key data science skills. Below are some general conclusions often derived from analyzing the Titanic dataset

- The dataset is a powerful tool for practicing data cleaning, visualization, feature engineering, and predictive modeling.
- Analysis of the Titanic dataset demonstrates how societal structures and situational factors can influence outcomes in crisis scenarios.

If you'd like, I can help craft a detailed conclusion or report specific to your analysis or provide assistance with visualizations and models to strengthen your conclusions!